Lecture 07 – Reinforcement Learning

Stat 5376: Statistical Machine Learning

Ruiqi Liu

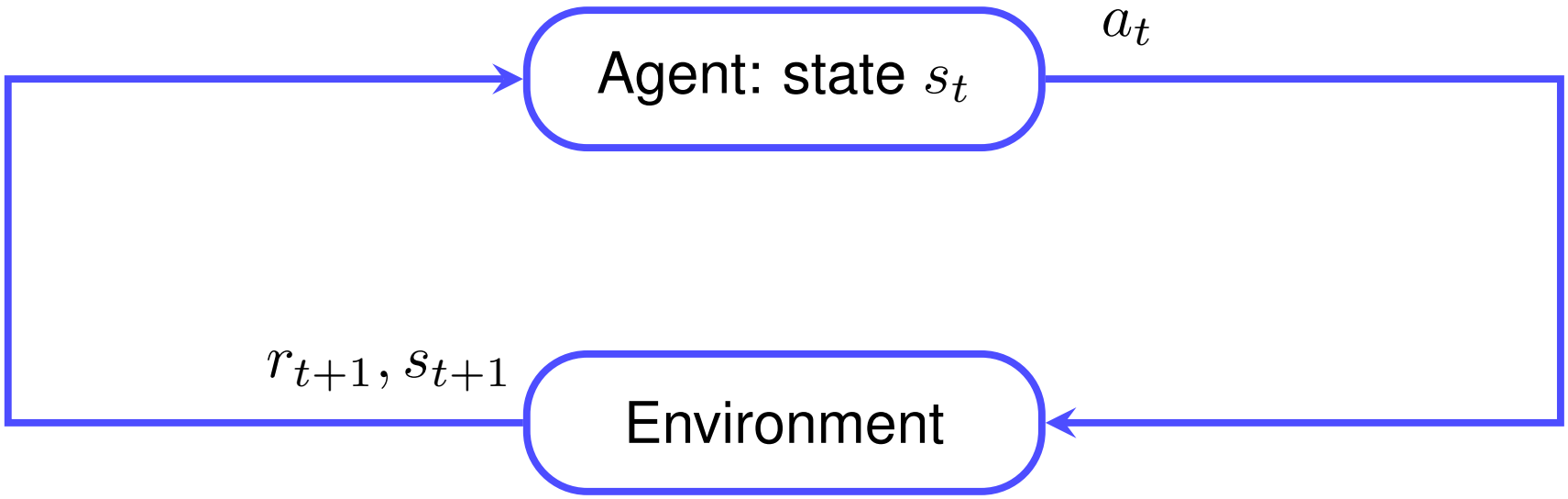
Department of Mathematics and Statistics
Texas Tech University

http://www.math.ttu.edu/~ruiqliu/

Disclaimer: These notes have not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor.

Markov Decision Process

- Model:



  - Current state: $s_t \in \mathcal{S}$
  - Action: $a_t \in \mathcal{A}$
  - Reward: $r_{t+1} \in \mathbb{R}$
  - Next state: $s_{t+1} \in \mathcal{S}$

Example

- Frozen lake: Find a path from S (start) to G (goal)

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

  - State set: $\mathcal{S} = \{1, 2, \ldots, 16\}$
  - Action set: $\mathcal{A} = \{\text{up, down, left, right}\}$
  - Reward: $r_{t+1} = 0$ if $s_{t+1} \neq 16$ and $r_{t+1} = 1$ if $s_{t+1} = 16$
- You also can define $r_{t+1} = -100$ if $s_{t+1} = 6, 8, 12, 13$ (H).

Mathematical Model

- Transition probability: $p(s' = s_{t+1} | s = s_t, a = a_t)$
  - Probaiblity of next state is s_{t+1} given current state is s_t and action is a_t
  - In some situations, next state is deterministic given current state and action
  - So $s' = s_{t+1}$ if $s = s_t$ and $a = a_t$ or $p(s' = s_{t+1} | s = s_t, a = a_t) = 1$
- Reward: $r_{t+1} = r(s_t, a_t, s_{t+1})$
  - Reward is a random variable given current state s_t, action a_t, and next state s_{t+1}
  - In many situations, reward only depends on the new state
  - So it becomes $r_{t+1} = r(s_{t+1})$
- (s_t, a_t, s_{t+1}): commonly used block

Example

- Vanilla frozen lake:

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

- Next state is derministic given current state and action:
  - $p(s' = 2 | s = 1, a = \text{right}) = 1$
  - $p(s' = 5 | s = 1, a = \text{down}) = 1$
- Reward only depends on new state:
  - $r(16) = 1, r(3) = 0$

Example

- Slippery frozen lake:

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

- The lake is slippery, so that the player may move perpendicular to the intended direction sometimes.
- Next state is random given current state and action:
  - $p(s' = 2|s = 1, a = \text{right}) = 0.5$, $p(s' = 5|s = 1, a = \text{right}) = 0.5$
  - $p(s' = 11|s = 7, a = \text{down}) = 1/3$, $p(s' = 6|s = 7, a = \text{down}) = 1/3$, $p(s' = 8|s = 7, a = \text{down}) = 1/3$

Policy

- A policy π is the agent's internal strategy on picking actions.
- Deterministic policies: maps state and action directly:

$$a_t = \pi(s_t).$$

- Stochastic policies: maps a probability of the action given a state:

$$\pi(a_t|s_t) = \text{ probability of choosing action } a_t \text{ given current state } s_t.$$

Example

- Frozen lake: Find a path from S (start) to G (goal)

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

- Detemrinistic policy:
  - $\pi(1) = \text{down}, \pi(5) = \text{down}, \pi(9) = \text{right}, \pi(10) = \text{right}, \pi(11) = \text{down}, \pi(15) = \text{right}$
  - For other state s, the policy $\pi(s) = \text{right}$.

Example

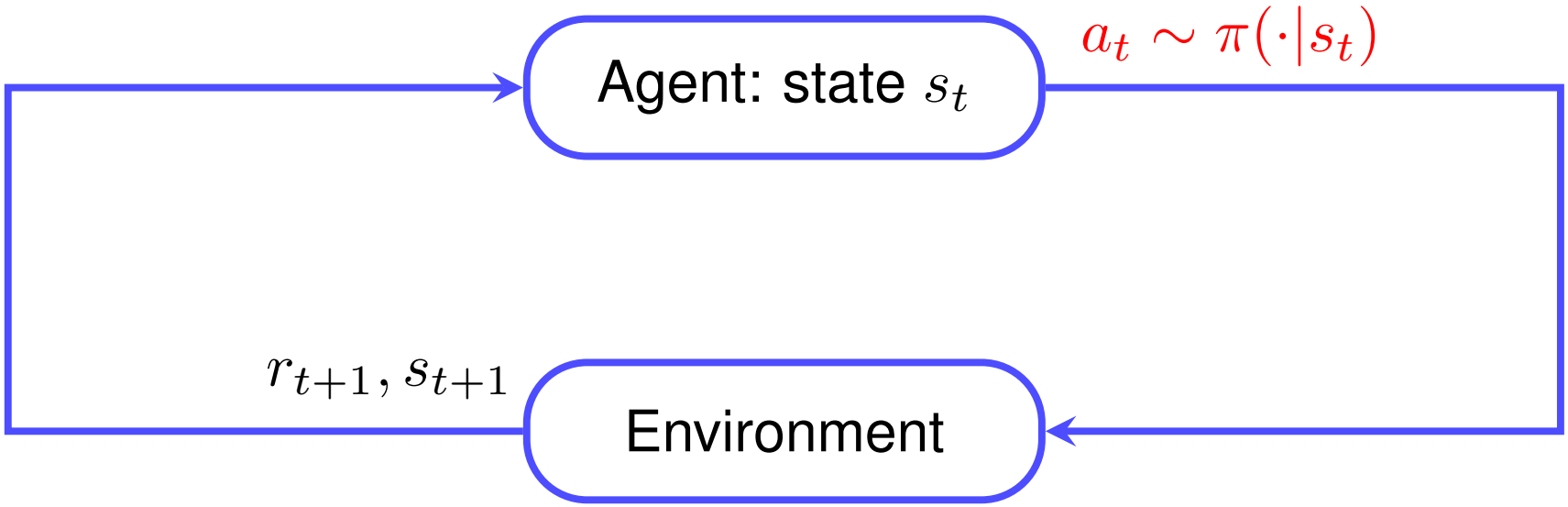
- Frozen lake: Find a path from S (start) to G (goal)

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

- Stochastic policy:
  - Uniform policy: $\pi(\text{up}|s) = \pi(\text{down}|s) = \pi(\text{right}|s) = \pi(\text{left}|s) = 1/4$

Markov Decision Process

- Model with a policy:



- Under the policy π, the agent will recieve rewards: $r_1, r_2, \ldots$.
- The total expected discounted reward is

$$\mathrm{E}_\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{k+1} \right\} = \mathrm{E}_\pi \left\{ r_1 + \gamma r_2 + \gamma^2 r_3 + \ldots \right\}$$

  - Discounted factor: $\gamma \in (0, 1)$
  - The rewards are random and depend on π, so we use E_π

Example

- Frozen lake: Find a path from S (start) to G (goal)

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

- Policy 1:
  - $\pi(1) = \text{down}, \pi(5) = \text{down}, \pi(9) = \text{right}, \pi(10) = \text{right}, \pi(11) = \text{down}, \pi(15) = \text{right}$
  - Total reward: $\mathrm{E}_\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{k+1} \right\} = \gamma^5$
- Policy 2:
  - $\pi(1) = \text{right}, \pi(2) = \text{right}, \pi(3) = \text{down}, \pi(7) = \text{down}, \pi(11) = \text{left}, \pi(10) = \text{down}, \pi(14) = \text{right}, \pi(15) = \text{right}$
  - Total reward: $\mathrm{E}_\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{k+1} \right\} = \gamma^8$

Evaluation Functions

- Value function:

$$V^{\pi}(s) = \mathrm{E}_{\pi}\left\{\sum_{k=0}^{\infty}\gamma^{k}r_{k+1}\middle| s_0 = s\right\}$$

- Under policy π and with initial state s, the value function $V^{\pi}(s)$ is the total expected discounted reward.
- Value-Action function:

$$Q^{\pi}(s,a) = \mathrm{E}_{\pi}\left\{\sum_{k=0}^{\infty}\gamma^{k}r_{k+1}\middle| s_0 = s, a_0 = a\right\}$$

- Under policy π and with initial state s, inital action a, the value-action function $Q^{\pi}(s,a)$ is the total expected discounted reward.
- Simple fact: $V^{\pi}(s) = \sum_a \pi(a|s)Q^{\pi}(s,a)$
  - Proof by the definition of conditional probaiblity and expectation
  - If π is derministic, then $V^{\pi}(s) = Q^{\pi}(s,\pi(s))$

Example

- Frozen lake: Find a path from S (start) to G (goal)

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

- Policy:
  - $\pi(1) = \text{down}, \pi(5) = \text{down}, \pi(9) = \text{right}, \pi(10) = \text{right}, \pi(11) = \text{down}, \pi(15) = \text{right}$
  - $V^\pi(1) = \gamma^5, V^\pi(11) = \gamma$
  - $Q^\pi(1, \text{down}) = Q^\pi(1, \pi(1)) = \gamma^5$, $Q^\pi(11, \text{left}) = \gamma^3$

Bellman Equation

- By the definition of Q^π, we have

$$
\begin{aligned}
Q^\pi(s,a) &= \mathrm{E}_\pi\left\{\sum_{k=0}^{\infty}\gamma^k r_{k+1}\middle| s_0 = s, a_0 = a\right\} \\
&= \mathrm{E}_\pi\left\{r_1 + \sum_{k=1}^{\infty}\gamma^k r_{k+1}\middle| s_0 = s, a_0 = a\right\} \\
&= \mathrm{E}_\pi\left\{r_1\middle| s_0 = s, a_0 = a\right\} + \mathrm{E}_\pi\left\{\sum_{k=1}^{\infty}\gamma^k r_{k+1}\middle| s_0 = s, a_0 = a\right\}
\end{aligned}
$$

- First item is easy:

$$
\begin{aligned}
\mathrm{E}_\pi\left\{r_1\middle| s_0 = s, a_0 = a\right\} &= \mathrm{E}_\pi\left\{r(s_0, a_0, s_1)\middle| s_0 = s, a_0 = a\right\} \\
&= \mathrm{E}_\pi\left\{r(s, a, s_1)\middle| s_0 = s, a_0 = a\right\} \\
&= \sum_{s'} p(s'|s,a) r(s,a,s')
\end{aligned}
$$

Bellman Equation

- Second iterm is tricky:

$$
\begin{aligned}
&\mathrm{E}_\pi\left\{\sum_{k=1}^{\infty}\gamma^k r_{k+1}\middle| s_0=s, a_0=a, s_1=s'\right\}\\
&=\gamma\mathrm{E}_\pi\left\{\sum_{k=1}^{\infty}\gamma^{k-1} r_{k+1}\middle| s_0=s, a_0=a, s_1=s'\right\}\\
&=\gamma\mathrm{E}_\pi\left\{\sum_{t=0}^{\infty}\gamma^{t} r_{t+2}\middle| s_0=s, a_0=a, s_1=s'\right\}\\
&=\gamma\mathrm{E}_\pi\left\{\sum_{t=0}^{\infty}\gamma^{t} r_{t+2}\middle| s_1=s'\right\}=\gamma V^\pi(s')
\end{aligned}
$$

- Hence, we have

$$
\mathrm{E}_\pi\left\{\sum_{k=1}^{\infty}\gamma^k r_{k+1}\middle| s_0=s, a_0=a\right\}=\sum_{s'}p(s'|s,a)\gamma V^\pi(s')
$$

Bellman Equation

- Bellman equation I:

$$Q^{\pi}(s,a) = \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma V^{\pi}(s')\right) \tag{1}$$

- Bellman equation II:

$$Q^{\pi}(s,a) = \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q^{\pi}(s',a')\right) \tag{2}$$

  - Prove using $V^{\pi}(s) = \sum_a \pi(a|s) Q^{\pi}(s,a)$

Optimal Policy

- Optimal policy:

$$V^{\pi^*}(s) \geq V^{\pi}(s), \text{ for all policy } \pi \text{ and all initial state } s \ .$$

  - π^* simultaneously maximizes $V^{\pi}(s)$ for all s.
  - Opitmal policy is not unique; see the frozen lake exmaple.
- Define optimal value functions:

$$V^*(s) = \max_{\pi} V^{\pi}(s), \quad Q^*(s,a) = \max_{\pi} Q^{\pi}(s,a)$$

  - The definition implies that $V^*(s) = V^{\pi^*}(s)$
  - But we only get $Q^{\pi^*}(s,a) \leq Q^*(s,a)$ at this moment, and will show that they are the same later!

Greedy Policy and Policy Improvement Theorem

- Given a policy π and its value-action function $Q^\pi(s,a)$, we can define a greedy policy as follows:

$$\pi'(s) = \operatorname*{argmax}_a Q^\pi(s,a)$$

- Example:

Table: $Q^\pi(s,a)$

		Action		
		1	2	3
State	1	1.3	**5.2**	-3
	2	**4.3**	3.5	2.3
	3	5.1	3.2	**8.8**
	4	-0.3	**4.2**	2.3

$$\pi'(1) = 2,$$
$$\pi'(2) = 1,$$
$$\pi'(3) = 3,$$
$$\pi'(4) = 2.$$

- Policy Improvement Theorem: $V^{\pi'}(s) \geq V^\pi(s)$ for all states s
  - Our goal is to find a policy π to maximize the value function $V^\pi(s)$
  - Policy improvement theorem tells we can always using the greedy policy to improve the value function: $\pi \to Q^\pi \to \pi'$
  - The proof is omitted

Optimal Value Functions Equation

- Goal: $V^*(s) = \max_a Q^*(s,a)$
- By the definition of V^* and Q^{π^*}, we have

$$\begin{aligned} V^*(s) = V^{\pi^*}(s) &= \sum_a \pi^*(a|s) Q^{\pi^*}(s,a) \\ &\overset{\text{(i)}}{\leq} \max_a Q^{\pi^*}(s,a) \overset{\text{(i)}}{\leq} \max_a Q^*(s,a) \end{aligned} \tag{3}$$

  - (i) is due to the fact that average is smaller than maximum
  - (ii) holds by the definition of Q^*

Optimal Value Functions Equation

- Consider a class of policies, Π_a, that commit action a at periord 0 (i.e., $a_0 = a$) regardless the state
- So we have

$$
\begin{aligned}
Q^*(s,a) = \sup_{\pi} Q^{\pi}(s,a) &= \sup_{\pi \in \Pi_a} Q^{\pi}(s,a) \\
&= \sup_{\pi \in \Pi_a} \mathrm{E}_{\pi}\left\{ \sum_{k=0}^{\infty} \gamma^k r_{k+1} \middle| s_0 = s, a_0 = a \right\} \\
&= \sup_{\pi \in \Pi_a} \mathrm{E}_{\pi}\left\{ \sum_{k=0}^{\infty} \gamma^k r_{k+1} \middle| s_0 = s, a_0 = \pi(s) \right\} \\
&= \sup_{\pi \in \Pi_a} V^{\pi}(s) \leq \sup_{\pi} V^{\pi}(s) = V^*(s).
\end{aligned}
$$

- Taking maximum over a, we have

$$
\max_a Q^*(s,a) \leq V^*(s).
$$

Optimal Value Functions Equation

- Using the previous two slides, we have

$$V^{\pi^*}(s) = V^*(s) = \max_a Q^*(s,a)$$

- Using the above equation and (3), it holds that

$$V^*(s) = V^{\pi^*}(s) \leq \max_a Q^{\pi^*}(s,a) \leq \max_a Q^*(s,a) = V^*(s)$$

- Optimal value functions equation:

$$V^{\pi^*}(s) = V^*(s) = \max_a Q^*(s,a) = \max_a Q^{\pi^*}(s,a) \tag{4}$$

- At this moment, we do not have $Q^*(s,a) = Q^{\pi^*}(s,a)$

Bellman Opitmality Equation

- By the definition of Q^* and (1), we have

$$
\begin{aligned}
Q^*(s,a) &= \max_{\pi} Q^{\pi}(s,a) \\
&= \max_{\pi} \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma V^{\pi}(s')\right) \\
&= \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma \max_{\pi} V^{\pi}(s')\right) \\
&= \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma V^{\pi^*}(s')\right) \qquad (5)
\end{aligned}
$$

- Taking maximum over a and using (4), we have Bellman opitmality equation I:

$$
V^{\pi^*}(s) = \max_{a} \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma V^{\pi^*}(s')\right) \qquad (6)
$$

Bellman Opitmality Equation

- Using (5) and (4), we have Bellman opitmality equation II:

$$Q^*(s,a) = \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma \max_a Q^*(s',a)\right) \tag{7}$$

- By (2) and (4), we have

$$\begin{aligned} Q^{\pi^*}(s,a) &= \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma V^{\pi^*}(s')\right) \\ &= \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma \max_a Q^*(s',a)\right) \end{aligned}$$

- Combining the previous two equations, we conclude that

$$Q^*(s,a) = Q^{\pi^*}(s,a)$$

Summary

- Bellman equation:

$$Q^{\pi}(s,a) = \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma V^{\pi}(s')\right),$$

$$Q^{\pi}(s,a) = \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q^{\pi}(s',a')\right).$$

- Optimal value functions equation:

$$Q^{\pi^*}(s,a) = Q^*(s,a),$$
$$V^{\pi^*}(s) = V^*(s) = \max_a Q^{\pi^*}(s,a) = \max_a Q^*(s,a).$$

- Bellman optimality equation:

$$V^{\pi^*}(s) = \max_a \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma V^{\pi^*}(s')\right),$$

$$Q^{\pi^*}(s,a) = \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma \max_a Q^{\pi^*}(s',a)\right).$$

Value Iteration

- Use Bellman equation to estimate Q^{π^*}:

$$Q^{\pi^*}(s,a) = \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma V^{\pi^*}(s')\right)$$

- Use Bellman opitmal equation to estimate V^{π^*}:

$$V^{\pi^*}(s) = \max_a \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma V^{\pi^*}(s')\right) = \max_a Q^{\pi^*}(s,a)$$

- Iteratively update Q^{π^*} and V^{π^*}

Value Iteration

Algorithm 1 Value Iteration

Require: Initialize $V(s) = 0$ for all $s \in \mathcal{S}$
Require: Choose an error tolerance $\epsilon > 0$ small
Require: Set initially $\Delta \leftarrow 1000000$ very large
  while $\Delta > \epsilon$ **do**
    for `all` s **do**
      for `all` a **do** # Bellman equation
        $Q(s, a) \leftarrow \sum_{s'} p(s'|s, a) \left(r(s, a, s') + \gamma V(s')\right)$
      end for
      $V_{\text{old}}(s) = V(s)$
      $V(s) \leftarrow \max_a Q(s, a)$ # Bellman optimality equation
    end for
    $\Delta \leftarrow \max_s |V_{\text{old}}(s) - V(s)|$
  end while
  Output policy $\pi^*(s) = \operatorname{argmax}_a Q(s, a)$

Policy Iteration

- Use Bellman equation to estimate Q^π:

$$Q^\pi(s,a) = \sum_{s'} p(s'|s,a)\left(r(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a')\right)$$

- Use policy improvement theorem to update π:

$$\pi' = \operatorname*{argmax}_a Q^\pi(s,a),$$
$$\pi = \pi'$$

- Iteratively use the previous two steps to update π

Policy Iteration

Algorithm 2 Policy Iteration

Require: Initialize policies $\pi(s) = 0$ for all $s \in \mathcal{S}$ and $\pi' \neq \pi$
Require: Initialize $Q(s, a) = 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$
Require: Choose an error tolerance $\epsilon > 0$ small

```
while π' ≠ π do
    Set initially Δ ← 1000000 very large
    while Δ > ε do # Bellman equation to estimate Q(s,a) = Q^π(s,a)
        for all (s,a) do
            Q_old(s,a) ← Q(s,a)
            Q(s,a) ← Σ_{s'} p(s'|s,a) (r(s,a,s') + γ Σ_{a'} π(a'|s') Q(s',a'))
        end for
        Δ ← max_{s,a} |Q_old(s,a) − Q(s,a)|
    end while
    for all s do
        π'(s) = π(s)
        π(s) = argmax_a Q(s,a) # policy improvement
    end for
end while
Output policy π(s)
```

Limitations of Value/Policy Iteration

- Need to know the transition probabilities $p(s'|s, a)$
- Need to know the complete information of the reward $r(s, a, s')$
- Are they reasonable?

Q-Learning

Algorithm 3 Q-Learning

Require: Initialize $Q(s, a) = 0$ for all s, a
Require: Some policy π_b (behavior policy)
Require: Learning rate α, number of epoches R
  for loop R times **do**
    Initial game and observe current state x
    while game is not over **do**
      Take action a based on s and behavior policy π_b
      Receive reward r and observe next state s'
      $Q(s, a) \leftarrow Q(s, a) + \alpha\big(r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big)$
      $s \leftarrow s'$
    end while
  end for
  Output policy $\pi^*(s) = \operatorname{argmax}_a Q(s, a)$

Behavior Policy π_b

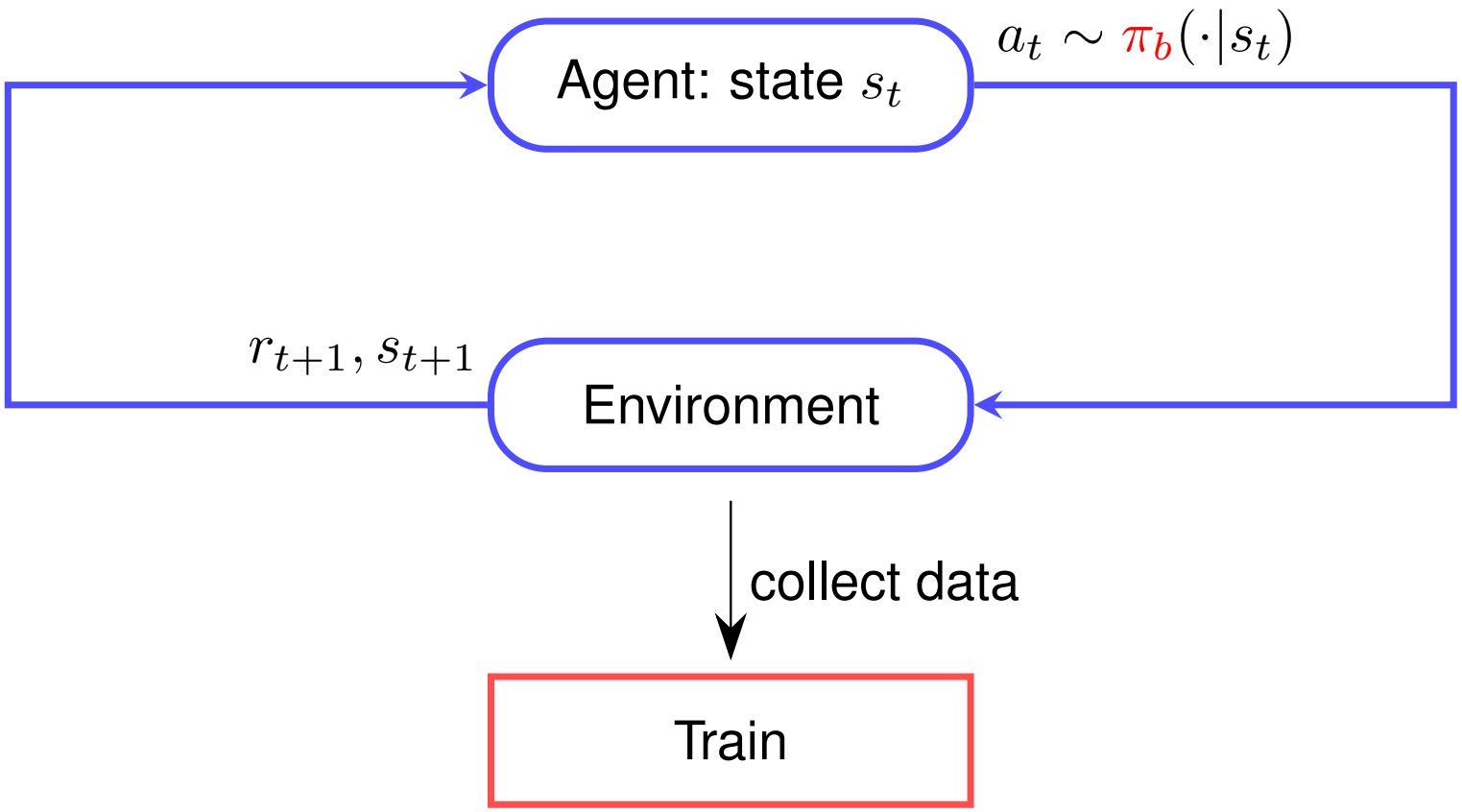
- Uniform Policy: Given any state s, the policy π_b has equal chance to choose each action.
- ϵ-greedy Policy: Given current Q-table, the optimal action given state s is $a_x^* = \mathrm{argmax}_a\, Q(s, a)$. The ϵ-greedy policy π_b given Q-table and current state s is

$$\begin{cases} a_x^* & \text{with probability } 1-\epsilon, \\ \text{randomly choose an action } a & \text{with probability } \epsilon. \end{cases}$$

- ϵ can be decreasing with the iterations.

Offline Reinforcement Learning

- Experiment is expensive, not everyone can interact with the envrioment.
- A person interact with the enviroment using a behavior policy π_b, and collect data: $(\text{current state}, \text{action}, \text{reward}, \text{next state})$
- Other people use the data to learn a policy.
- Work flow:



Offline Q-Learning

Algorithm 4 Offline Q-Learning

Require: Fixed dataset $\mathcal{D} = \{(s_t, a_t, r_t, s'_t)\}_{t=1}^{N}$, here $s'_t = s_{t+1}$
Require: Initialize $Q(s, a) = 0$ for all s, a
Require: Learning rate α, discount factor γ, number of epoches R
  for epoch $= 1, \ldots, R$ **do**
    for each transition $(s, a, r, s') \in \mathcal{D}$ **do**
      $Q(s, a) \leftarrow Q(s, a) + \alpha\Big(r + \gamma \max_{a'} Q(s', a') - Q(s, a)\Big)$
    end for
  end for
  Output policy $\pi^*(s) = \arg\max_a Q(s, a)$

Coding

- See instruction in class

More about Q-Learning

- Limitations of Q-table based methods
  - Infeasible when state space or action space is large, see Pong example
  - DQN: Use neural network $Q_\theta(s, a)$ to approximate Q-table

Policy Gradient Method

- Q-table based methods:
  - Find the optimal Q function: $Q^*(s, a)$
  - Find the greedy policy: $\pi^*(s) = \mathrm{argmax}_a \, Q^*(s, a)$
- Gradient based methods:
  - Parametrize the policy π_θ
  - Use gradient method updating θ to find the opitmal value of

$$J(\theta) = \sum_s \rho_0(s) V^{\pi_\theta}(s)$$

  - $\rho_0(s)$ is the probablity of inital state $s_0 = s$
  - In the frozen like exmaple, $\rho_0(1) = 1$ as it always starts at $s_0 = 1$.
- Softmax Policy:

$$\pi_\theta(a|s) = \frac{e^{\theta_{s,a}}}{\sum_{a'} e^{\theta_{s,a'}}} \text{ with } \theta = (\theta_{s,a}) \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{A}|}$$

Policy Gradient Method

- Suppose $s_0 \sim \rho_0$ and we have a trajectory

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots) \text{ generated from policy } \pi_\theta$$

- We can find the distribution

$$\begin{aligned} p_\theta(\tau) &= \rho_0(s_0)\pi_\theta(a_0|s_0)p(s_1|s_0,a_0) \times \pi_\theta(a_1|s_1)p(s_2|s_1,a_2) \times \ldots \\ &= \rho_0(s_0)\prod_{t=0}^{\infty}\pi_\theta(a_t|s_t)p(s_{t+1}|s_t,a_t) \end{aligned}$$

- Hence the value functions is

$$J(\theta) = \sum_s \rho_0(s)V^{\pi_\theta}(s) = \sum_\tau p_\theta(\tau)\sum_{k=0}^{\infty}\gamma^k r_{k+1} = \sum_\tau p_\theta(\tau)R(\tau)$$

Policy Gradient Method

- Now we have

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{\infty} \pi_\theta(a_t|s_t) p(s_{t+1}|s_t, a_t),$$

$$J(\theta) = \sum_\tau p_\theta(\tau) R(\tau)$$

- Use the traick $\nabla p_\theta = p_\theta \nabla \log p_\theta$, we have

$$\begin{aligned}
\nabla J(\theta) &= \sum_\tau R(\tau) p_\theta(\tau) \nabla \log p_\theta(\tau) \\
&= \sum_\tau R(\tau) p_\theta(\tau) \sum_{t=0}^{\infty} \nabla \log \pi_\theta(a_t|s_t) \\
&= \mathrm{E}_{\pi_\theta} \left\{ R(\tau) \sum_{t=0}^{\infty} \nabla \log \pi_\theta(a_t|s_t) \right\}
\end{aligned}$$

Policy Gradient Method

- If $k < t$, we have

$$\mathrm{E}_{\pi_\theta}\left\{r_k \nabla \log \pi_\theta(a_t|s_t)\right\} = \mathrm{E}_{\pi_\theta}\left\{r(s_k, a_k, s_{k+1}) \nabla \log \pi_\theta(a_t|s_t)\right\} \overset{\text{(i)}}{=} 0$$

- To verify (i), we condition on $(s_0, a_0, r_0, s_1, \ldots, s_t)$. Therefore, it holds that

$$\begin{aligned}
&\mathrm{E}_{\pi_\theta}\left\{r(s_k, a_k, s_{k+1}) \nabla \log \pi_\theta(a_t|s_t) \middle| s_0, a_0, r_0, s_1, \ldots, s_t\right\} \\
&= r(s_k, a_k, s_{k+1}) \mathrm{E}_{\pi_\theta}\left\{\nabla \log \pi_\theta(a_t|s_t) \middle| s_0, a_0, r_0, s_1, \ldots, s_t\right\} \\
&= r(s_k, a_k, s_{k+1}) \mathrm{E}_{\pi_\theta}\left\{\nabla \log \pi_\theta(a_t|s_t) \middle| s_t\right\} \\
&= r(s_k, a_k, s_{k+1}) \sum_{a_t} \pi_\theta(a_t|s_t) \nabla \log \pi_\theta(a_t|s_t) \\
&= r(s_k, a_k, s_{k+1}) \sum_{a_t} \nabla \pi_\theta(a_t|s_t) \\
&= r(s_k, a_k, s_{k+1}) \nabla \sum_{a_t} \pi_\theta(a_t|s_t) = r(s_k, a_k, s_{k+1}) \nabla 1 = 0
\end{aligned}$$

Policy Gradient Method

- Hence, we have

$$\begin{aligned}\nabla J(\theta) &= \mathrm{E}_{\pi_\theta}\left\{R(\tau)\sum_{t=0}^{\infty}\nabla\log\pi_\theta(a_t|s_t)\right\}\\ &= \mathrm{E}_{\pi_\theta}\left\{\sum_{k=0}^{\infty}\gamma^k r_{k+1}\sum_{t=0}^{\infty}\nabla\log\pi_\theta(a_t|s_t)\right\}\\ &= \mathrm{E}_{\pi_\theta}\left\{\sum_{t=0}^{\infty}\nabla\log\pi_\theta(a_t|s_t)\sum_{k=t}^{\infty}\gamma^k r_{k+1}\right\}\end{aligned}$$

- Policy gradient (reward-to-go version):

$$\nabla J(\theta) = \mathrm{E}_{\pi_\theta}\left\{\sum_{t=0}^{\infty}\nabla\log\pi_\theta(a_t|s_t)G_t\right\},\ \text{with } G_t = \sum_{k=t}^{\infty}\gamma^k r_{k+1}$$

Policy Gradient

Algorithm 5 Policy Gradient

Require: Initialize policy parameter θ
Require: Learning rate α, number of epoches R
  for loop R times **do**
    Initialize game and observe current state s_0
    Generate one episode $(s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T)$ using policy $\pi_\theta(a \mid s)$
    for $t = 0, 1, \ldots, T-1$ **do**
      Compute return $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_{k+1}$
      $\theta \leftarrow \theta + \alpha\, G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$
    end for
  end for
  Output policy $\pi_\theta(a \mid s)$

Offline Policy Gradient

- Suppose $s_0 \sim \rho_0$ and we have a trajectory

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots) \text{ generated from a behavior policy } \pi_b$$

- We still have

$$\begin{aligned}\nabla J(\theta) &= \mathrm{E}_{\pi_\theta}\left\{\sum_{t=0}^{\infty} \nabla \log \pi_\theta(a_t|s_t) G_t\right\} \\ &= \mathrm{E}_{\pi_b}\left\{\sum_{t=0}^{\infty}\left(\prod_{u=0}^{t} \frac{\pi_\theta(a_u|s_u)}{\pi_b(a_u|s_u)}\right) \nabla \log \pi_\theta(a_t|s_t) G_t\right\}\end{aligned}$$

  - Importance sampling:

$$\mathrm{E}(f(X)) = \int f(x) p_X(x) dx = \int f(x) \frac{p_X(x)}{p_Y(x)} p_Y(x) dx = \mathrm{E}\left(\frac{p_X(Y)}{p_Y(Y)} f(Y)\right)$$

Offline Policy Gradient

Algorithm 6 Offline Policy Gradient

Require: Initialize policy parameter θ
Require: Offline dataset $\mathcal{D} = \{\tau^{(i)}\}_{i=1}^{n}$ collected under behavior policy π_b
Require: Learning rate α, discount factor γ, number of epoches R

```
for loop R times do
    Sample one episode (s_0, a_0, r_1, s_1, a_1, r_2, ..., s_{T-1}, a_{T-1}, r_T) from D
    for t = 0, 1, ..., T-1 do
        Compute return G_t = sum_{k=t}^{T-1} γ^{k-t} r_{k+1}
        Compute importance weight W_t = prod_{u=0}^{t} π_θ(a_u | s_u) / π_b(a_u | s_u)
        θ ← θ + α W_t G_t ∇_θ log π_θ(a_t | s_t)
    end for
end for
Output policy π_θ(a | s)
```

Coding

- See instruction in class

More on Policy Gradient

- Policy gradient (reward-to-go version):

$$\nabla J(\theta) = \mathrm{E}_{\pi_\theta}\left\{\sum_{t=0}^{\infty} \nabla \log \pi_\theta(a_t|s_t)(G_t - b(s_t))\right\}$$

  - The equation holds for any functioin $b(s)$
  - Because $\mathrm{E}_{\pi_\theta}\left\{\nabla \log \pi_\theta(a_t|s_t) b(s_t)\right\} = 0$
- A good choice is $V^{\pi_\theta}(s_t)$
  - This motivates the Actor-critic methods
  - Actor to update θ
  - Critic to update $V_w(s) \approx V^{\pi_\theta}(s)$ using an additional parameter w

Offline Reinforcement Learning

- Offline Q-leanring:
  - Do not need to know the expression of π_b
- Offline policy gradient:
  - Need to know the expression of π_b
- Is this assumption feasible?